

I. THE BIOLOGICAL INSPIRATION

Artificial Neural Networks (ANN) are loosely inspired by the human brain's network of neurons. In the brain, a neuron receives electrical signals, and if the signal is strong enough, it "fires" to the next neuron. In AI, we mimic this using mathematical functions.

1.1. The Artificial Neuron (The Perceptron)

The Perceptron is the simplest form of a neural network. It takes several inputs ($x_1, x_2, \dots$), multiplies them by **Weights** ($w_1, w_2, \dots$), adds a **Bias** (b), and passes the result through an **Activation Function**.

II. ANATOMY OF A NEURAL NETWORK

A modern "Deep" neural network consists of three main types of layers:

1. **Input Layer:** Receives the raw data (e.g., pixels of an image or words in a sentence).
2. **Hidden Layers:** Where the "learning" happens. A network is called "Deep" if it has more than one hidden layer. These layers extract increasingly complex features.
3. **Output Layer:** Produces the final prediction (e.g., "90% probability this is a cat").

III. HOW THE NETWORK LEARNS: THE 3-STEP CYCLE

Learning in a neural network is an iterative process of trial and error.

Step 1: Forward Propagation

Data flows from the input to the output. Each neuron calculates a weighted sum and applies an activation function. The final output is compared to the **True Label** using a **Loss Function** (e.g., Mean Squared Error).

Step 2: Loss Calculation

The Loss Function measures "how wrong" the network is. If the network predicts "Dog" but the image is a "Cat," the loss will be high. The goal of training is to minimize this loss.

Step 3: Backpropagation (The "Magic" of AI)

This is the most important algorithm in AI. Backpropagation calculates how much each weight contributed to the error. It uses the **Chain Rule (Calculus)** to send the error signal backward through the network, from the output layer to the input layer.

IV. OPTIMIZATION: GRADIENT DESCENT

Once we know which weights are responsible for the error, we need to update them. We use **Gradient Descent**.

- Imagine you are on a foggy mountain and want to find the lowest valley. You take small steps in the direction where the ground slopes downward.

- **Learning Rate (η):** The size of the steps. If the rate is too high, you might jump over the valley. If it's too low, it will take forever to reach the bottom.

V. ACTIVATION FUNCTIONS: ADDING NON-LINEARITY

Without activation functions, a neural network is just a giant linear equation, which cannot solve complex problems.

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. The most popular function for hidden layers because it is fast and prevents mathematical errors.
- **Sigmoid:** Maps values between 0 and 1. Used for binary classification (Yes/No).
- **Softmax:** Used in the output layer for multi-class problems (e.g., identifying digits 0-9).

VI. PRACTICAL LAB: BUILDING A SIMPLE NEURON (PYTORCH)

PyTorch is the industry-standard library for Deep Learning at DTU.

Python

```
import torch
```

```
import torch.nn as nn
```

1. Define a simple Neural Network with 1 Hidden Layer

```
class SimpleNN(nn.Module):
```

```
    def __init__(self, input_size, hidden_size, output_size):
```

```
        super(SimpleNN, self).__init__()
```

```
        self.hidden = nn.Linear(input_size, hidden_size) # Weight + Bias
```

```
        self.relu = nn.ReLU() # Activation
```

```
        self.output = nn.Linear(hidden_size, output_size)
```

```
    def forward(self, x):
```

```
        x = self.hidden(x)
```

```
        x = self.relu(x)
```

```
        x = self.output(x)
```

```
        return x
```

2. Initialize Model

```
model = SimpleNN(input_size=10, hidden_size=5, output_size=1)
```

```
print(model)
```